

Faculty of Computers and Information Systems

Information Retrieval

Practical Laboratory

Week 9: Evaluation of Information Retrieval Systems

Evaluate the Week 8 ranking against explicit relevance judgments. Compare a scored ranking and a filename-order baseline using the same queries and cutoffs.

Duration: approximately 2 practical hours. Suggested pacing: 15 minutes concept review, 25 demonstration, 50 guided implementation, 20 selected exercises, and 10 review. Finish remaining exercises after the session.

1. Lab Objectives

- Identify true positives, false positives, and false negatives.
- Implement precision, recall, F1, Precision@k, and Recall@k.
- Evaluate computed rankings against predefined relevance sets.
- Compare two outputs fairly and state the limits of the comparison.

2. Required Background

Prerequisite: basic Python variables, conditions, loops, lists, dictionaries, sets, and functions. Complete the earlier labs before starting this week.

Retrieved documents are the IDs returned under the chosen retrieval policy. Relevant documents are IDs judged useful for a particular information need. Relevance is not inferred from a score. The JSON file defines three queries, their intents, and their relevant document IDs for this teaching collection.

TP counts retrieved IDs in the relevance set. FP counts retrieved IDs outside it. FN counts relevant IDs not retrieved. Precision asks what fraction of retrieved documents is relevant; recall asks what fraction of all relevant documents was retrieved. F1 is their harmonic mean.

This lab defines a zero result for precision when nothing is retrieved, recall when the relevance set is empty, and F1 when precision plus recall is zero. These are reporting conventions for undefined ratios; an empty relevance set should be flagged when interpreting an evaluation.

Precision@k is the number of relevant unique IDs in the first k positions divided by k. If fewer than k results are returned, unfilled positions count as nonrelevant. Recall@k divides the same relevant count by the total number of relevant IDs. k must be positive. Duplicate IDs are removed while keeping their first appearance.

Both methods use k = 3. The TF-IDF method returns only positive-score documents from the Week 8 ranking; the baseline orders all documents by filename. Ordinary precision/recall/F1 in the report are calculated on the available top-three IDs. P@3 can differ from ordinary precision when fewer than three results exist.

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$Precision@k = \frac{\text{relevant IDs in top } k}{k}$$

$$Recall@k = \frac{\text{relevant IDs in top } k}{\text{total relevant IDs}}$$

Course material: Rankings are calculated by retrieval.py, an unchanged copy of the Week 8 program. Relevance judgments are teaching examples supplied with this package.

3. Python Concepts Used

- **Set intersection and difference:** Count TP, FP, and FN using document IDs.
- **Slicing:** Take the first k entries of a ranked sequence.
- **JSON:** Read explicit query intents and relevance sets.
- **Guard clauses:** Handle zero denominators and reject nonpositive k.
- **Module imports:** Reuse Week 8 retrieval code instead of manually entering scores.

4. Worked Example

Use the same relevance set for both outputs: relevant = {d1, d3, d5}. Output A is [d1, d2, d3]; output B is [d1, d4, d2]. Calculate the metrics before executing the example.

```
relevant = {"d1", "d3", "d5"}
outputs = {"A": ["d1", "d2", "d3"],
           "B": ["d1", "d4", "d2"]}
for label, retrieved in outputs.items():
    tp = len(set(retrieved) & relevant)
    p = tp / len(retrieved)
    r = tp / len(relevant)
    f1 = 0 if p + r == 0 else 2 * p * r / (p + r)
    print(label, f"P={p:.4f} R={r:.4f} F1={f1:.4f}")
```

Output A has TP=2, FP=1, FN=1 and P=R=F1=0.6667. Output B has TP=1, FP=2, FN=2 and P=R=F1=0.3333. A performs better on this particular judged example. It does not establish universal superiority across other queries, collections, or relevance definitions.

Query	Relevant IDs	Information need
python data analysis	doc1, doc4, doc9	Python or analysis in a data-science context
information retrieval documents	doc8, doc10	Document retrieval and search evaluation
database queries tables	doc6	Relational database organization

5. Step-by-Step Practical Task

Step 1: Review each query intent and relevance set in data/relevance.json before looking at the scores.

Step 2: Run the Week 8 retrieval implementation copied into retrieval.py; keep all ten documents unchanged.

Step 3: Retain only positive-score IDs and take the first three. Independently count TP, FP, and FN.

Step 4: Implement the metric functions and compare your values with the program.

Step 5: Evaluate the filename-order baseline using the same relevance sets and k.

Step 6: Explain any difference between precision and P@3 when the ranking has fewer than three positive results. Report findings per query and avoid universal claims.

Execution: open a terminal in this week folder and run python lab09_solution.py. On Windows, py -3 can replace python. All required files are included; no internet or third-party Python packages are needed.

Keep retrieval.py beside lab09_solution.py. It is the complete Week 8 program, copied unchanged; its main guard prevents the demonstration from running during import. Its full listing is in the Week 8 sheet.

6. Complete Python Program

The listing below matches lab09_solution.py exactly. Keep indentation when copying. The exercise starter is separate and intentionally unfinished.

lab09_solution.py

```
import json
import os
from retrieval import load_documents, build_tfidf, rank_query

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def precision(retrieved, relevant):
    retrieved = set(retrieved)
    relevant = set(relevant)
    if not retrieved:
        return 0.0
    return len(retrieved & relevant) / len(retrieved)

def recall(retrieved, relevant):
    relevant = set(relevant)
    if not relevant:
        return 0.0
    return len(set(retrieved) & relevant) / len(relevant)

def f1_score(p, r):
    if p + r == 0:
        return 0.0
    return 2 * p * r / (p + r)

def unique_ranking(ranked_docs):
    result = []
    for name in ranked_docs:
        if name not in result:
            result.append(name)
    return result

def precision_at_k(ranked_docs, relevant_docs, k):
    if k <= 0:
        raise ValueError("k must be positive")
    top = unique_ranking(ranked_docs)[:k]
    # Unfilled positions count as nonrelevant: denominator stays k.
    return len(set(top) & set(relevant_docs)) / k

def recall_at_k(ranked_docs, relevant_docs, k):
    if k <= 0:
        raise ValueError("k must be positive")
    return recall(unique_ranking(ranked_docs)[:k], relevant_docs)

def report(label, ranked_docs, relevant, k=3):
    top = unique_ranking(ranked_docs)[:k]
    true_positive = len(set(top) & set(relevant))
    false_positive = len(set(top) - set(relevant))
    false_negative = len(set(relevant) - set(top))
    p = precision(top, relevant)
    r = recall(top, relevant)
    print(label, "top:", top)
    print("TP, FP, FN:", true_positive, false_positive, false_negative)
    print(f"Precision={p:.4f} Recall={r:.4f} F1={f1_score(p, r):.4f}")
    print(f"P@{k}={precision_at_k(ranked_docs, relevant, k):.4f}")
    print(f"R@{k}={recall_at_k(ranked_docs, relevant, k):.4f}")
```

```
def main():
    documents = load_documents(os.path.join(BASE_DIR, "docs"))
    vocabulary, counts, df, idf, vectors = build_tfidf(documents)
    path = os.path.join(BASE_DIR, "data", "relevance.json")
    with open(path, "r", encoding="utf-8") as file:
        judgments = json.load(file)
    for case in judgments:
        query = case["query"]
        relevant = set(case["relevant"])
        results = rank_query(query, vocabulary, idf, vectors)
        ranked = [name for name, score in results if score > 0]
        baseline = sorted(documents) # Filename order, not relevance.
        print("\nQuery:", query)
        print("Relevant:", sorted(relevant))
        report("TF-IDF", ranked, relevant)
        report("Filename baseline", baseline, relevant)

if __name__ == "__main__":
    main()
```

7. Expected Output

Representative output from the shipped collection and the default command. Calculated values come from a tested program run. Long output lines wrap here for printing; expected_output.txt contains the full terminal output. Selected rows are shown for large matrices.

```
Query: python data analysis
Relevant: ['doc1', 'doc4', 'doc9']
TF-IDF top: ['doc4', 'doc9', 'doc1']
TP, FP, FN: 3 0 0
Precision=1.0000 Recall=1.0000 F1=1.0000
P@3=1.0000
R@3=1.0000
Filename baseline top: ['doc1', 'doc10', 'doc2']
TP, FP, FN: 1 2 2
Precision=0.3333 Recall=0.3333 F1=0.3333
P@3=0.3333
R@3=0.3333

Query: information retrieval documents
Relevant: ['doc10', 'doc8']
TF-IDF top: ['doc8', 'doc10']
TP, FP, FN: 2 0 0
Precision=1.0000 Recall=1.0000 F1=1.0000
P@3=0.6667
R@3=1.0000
Filename baseline top: ['doc1', 'doc10', 'doc2']
TP, FP, FN: 1 2 1
Precision=0.3333 Recall=0.5000 F1=0.4000
P@3=0.3333
R@3=0.5000

Query: database queries tables
Relevant: ['doc6']
TF-IDF top: ['doc6']
TP, FP, FN: 1 0 0
Precision=1.0000 Recall=1.0000 F1=1.0000
P@3=0.3333
R@3=1.0000
Filename baseline top: ['doc1', 'doc10', 'doc2']
TP, FP, FN: 0 3 1
Precision=0.0000 Recall=0.0000 F1=0.0000
P@3=0.0000
R@3=0.0000
```

These results use all ten shipped documents. Editing the collection changes the index and may change weights, ranks, and evaluation results.

8. Laboratory Exercises

1. Hand calculation: finish the A/B example and compute $P@2$ and $R@2$ for each output.
2. Implementation: complete `precision_at_k` in the starter using the documented short-list convention.
3. Debugging: explain why dividing recall by the number of retrieved documents gives the wrong measure.
4. Interpretation: test an empty retrieved list, an empty relevant set, and k larger than the ranking.
5. Comparison: evaluate both outputs for all three supplied queries with identical relevance sets.
6. Extension: compare $k=1$, $k=3$, and $k=5$ on the same scored ranking. Explain the precision/recall tradeoff.
7. Data quality: introduce a repeated document ID and show how deduplication prevents it from inflating a metric.

9. Student Tasks

- Completed metric functions and hand-checked TP/FP/FN counts for each query.
- A table comparing both methods under the same relevance set and cutoff.
- A short conclusion limited to this collection, these judgments, and the chosen cutoff.

Submit your own implementation and evidence. Do not submit an untouched starter as a completed solution. Label all outputs with the query or input that produced them.

10. Review Questions

1. What distinguishes relevance from similarity?
2. What is a false negative?
3. Why can precision and recall move in opposite directions?
4. How does this lab define $P@k$ when fewer than k results exist?
5. Why does one successful query not prove that a retrieval method is always better?

11. Challenge Exercise

Optional: Compute macro-averaged precision, recall, and F1 over the three queries. Compare mean per-query F1 with F1 calculated from mean precision and mean recall; explain why they need not match.